

Part Two

Part A.

These are based on a static image of the screen, so some points are things I'd verify by testing rather than confirmed bugs, anything about what happens on submission (validation, error messages, double-click behavior) can't be seen in a mock-up.

Payment Fields

1. There's no security code field CVV/CVC

Every card payment normally asks for a 3-digit code on the back of the card. It isn't here. This is the one I'd call most severe, it's the main check that the person actually holds the card rather than just knowing the number, so without it fraud is much easier.

2. Card Type is a dropdown the user must set

The card number itself already says which type it is, Visa numbers start with 4, MasterCard with 5. Asking the user to pick allows creating a mistake that shouldn't be possible.

Example: select VISA, type a MasterCard number and get a confusing rejection.

Also, the dropdown defaults to VISA, so it's never empty. The asterisk marking it required has no effect, and a user paying with a Mastercard can submit the form without ever noticing they left it on the wrong card type.

3. Card Number (No dashes or spaces)

The number, if type Visa for example, is printed on the card in groups of four, so the form is asking the user to enter it differently from how they're reading it. Software can strip spaces in one line of code. It is making the user do the machine's work.

Global Problems

The question says this is a **global** SaaS company, which makes these serious.

1. "Payment Amount 30.00" with no currency and nothing says what the 30.00 is for.

Thirty of what? Dollars, ILS, Euros, Pounds? The customer has no way to know what they're agreeing to pay. Also, No plan name, no billing period.

2. "State or Province" → "Select a state"

A dropdown of US states. A customer in Israel for example has nothing to select, and the field is marked required, so they may be unable to complete payment at all if it's not adjusted to regions.

3. No Country field

Nothing on the form asks where the customer is from. There is a “City” field but no “Country” field. Without a country, the form can't validate the postal code or show the right regional field.

4. Postal Code (no dashes)

Again, asking the user to do the work of “no dashes” where the system is supposed to do it. Also, since there is no country field and the postal code pattern depends on the country, there is no real way to validate the postal code entered.

5. First Name, MI, Last Name.

“MI” means Middle Initial, which is a US convention. Many people worldwide don't have a name that splits this way. A single 'Name on card' field, matching what's printed on the card, would be simpler.

Usability

1. “Continue” doesn't say what happens

Will clicking it charge the card, or show a review step first? On a payment screen the user needs to know before clicking.

2. The second address line has no label

Just an unlabeled box below the street address. The user has no idea what it's for.

3. No visible place (space) for error messages

If the card is declined, where does that appear?

4. No trust signals

No padlock, no mention that the connection is secure. On a payment form, users look for this.

Things I'd need to test before assuming bugs

The first two are covered by the test cases in part B.

1. Double-clicking Continue

If the user clicks twice on a slow connection, are they charged twice? Nothing suggests protection against it. Worth testing whether a second click sends a second charge.

2. **Month and Year** are separate dropdowns. I'd test whether a date in the past is rejected.
3. After filling in ten fields, an **accidental "cancel" click** would lose everything. I'd test what actually happens, and would expect a confirmation step before discarding entered data.

Part B.

Test #1

Successful payment with valid card details

Priority: High

Preconditions: User is on the Account Information screen with a payment amount of 30.00.

1. Select "VISA" from Card Type
2. Enter a valid Visa test card number with no spaces
3. Enter First Name and Last Name
4. Enter street address, city, and select a state (US)
5. Enter a valid postal code (US)
6. Select a future month and year for Expiration
7. Click Continue

Expected: The payment is accepted, the user is taken to the next step, and the charge of 30.00 appears once on the account. All required fields marked with * must be filled for the form to submit.

Why this case: Sanity test that the basic payment flow works.

Test #2

Double-click on Continue

Priority: High

Preconditions: Same as above.

1. Fill in all required fields with valid values
2. Click Continue twice in quick succession

Expected: The customer is charged 30.00 once. The button should become disabled after the first click so a second request can't be sent.

Why this case: On a slow connection a user who doesn't see a response may click again. A double charge means a refund, a support ticket, and a customer who doesn't trust the product.

Test #3

Customer outside the United States

Priority: High

Preconditions: Same as above.

1. Fill in the card details with a valid card
2. Enter a street address and city in Israel
3. Attempt to complete the "State or Province" field
4. Enter an Israeli postal code (7 digits)
5. Click Continue

Expected: The customer is supposed to be able to complete and submit the form. Currently expected to fail presuming that "Select a state" offers only US states and is a required field, so a non-US customer may be unable to pay at all.

Notes on these choices:

I picked these three by what it costs when each one fails, rather than by what is easiest to test.

Test #1 comes first because it's the sanity check: if a valid card can't complete a payment, nothing else on the screen matters.

Test #2 costs real money. A double charge means a refund, a support ticket, and a customer who no longer trusts the product. It's also the kind of thing that only happens on a slow connection, so it can easily reach production unnoticed.

Test #3 is about customers who can't pay at all. The question describes a global company, and if the address section only works for US customers, everyone else is blocked with no workaround. That's lost revenue rather than an inconvenience.

The **cases below** matter, but each one fails less expensively. A past expiry date or a card type mismatch produces a confusing error and the customer tries again. A missing required setting lets through incomplete data, which usually surfaces later as a failed payment rather than a wrong one. Having to remove spaces from a card number is friction, not a blocker. None of them stop a customer paying or take money twice, which is why they come after the first three.

I also chose one passing case and two failing ones deliberately. Three tests that all pass would confirm very little.

Other cases I would add:

- Enter a **Month and Year** in the **past** and check the payment is rejected.
- Select **VISA** but enter a **Mastercard** number, and check the **mismatch** is caught with a clear message rather than a generic rejection.
- Submit with a **required field empty** and check the **form is blocked** with a clear message.
- Enter a **card number** with spaces, as printed on the card, and check it is either accepted or the spaces removed automatically.

Part C.

The most severe bug: there is no CVV field.

The CVV is the 3-digit code on the back of the card. Its purpose is to check that the person paying is actually holding the card, rather than just knowing the number. The CVV isn't stored by shops, so a thief with a stolen number usually doesn't have it.

Without this field, the form accepts payments from anyone who has a card number. That makes fraud much easier. I ranked it above the problems that stop customers outside the US from paying, because a customer who can't pay will contact support or give up, but a fraudulent payment means real money is taken from a real person, followed by a chargeback and the cost of handling it.

The immediate fix is to add a CVV field and require it.

The bigger problem behind it

Most of the issues I listed have the same cause: this company built its own payment form.

That means the company is responsible for card security, detecting the card type, formatting card numbers, handling addresses in every country, and checking postal codes against each country's rules. The mock-up shows several of those going wrong at once. Fixing them one at a time means fixing each of them again in future, every time a new country or card type comes along.

The product solution

Use a payment provider such as Stripe or PayPal for the card section, instead of building it. These companies provide a card form that sits inside the page. To the customer it looks the same as it does now: they type their card number, expiry date and CVV in the usual place. The details go to the provider rather than to this company's own servers, and the company receives back a token (a code representing the card) which it uses to take the payment.

This solves most of my list at once, because it's the provider's job to get these right:

- The CVV field is included, since the provider requires it
- The card type is detected from the number, so the “Card Type” dropdown isn't needed at all
- Spaces in the card number are handled automatically, so “no dashes or spaces” disappears
- The address section changes based on the country, “State” for a US customer, something appropriate elsewhere
- Postal codes are checked using the right rules for that country
- The provider's name on the form is itself a sign to the customer that payment is handled securely

There's also a security benefit beyond the CVV. If card details never reach this company's own servers, the company can't lose them. Storing card data safely is difficult and heavily regulated, so not storing it at all removes a large risk.

What it costs

- The provider charges a fee on every payment
- Less freedom over how the form looks, though it can be styled to match
- The company now depends on another company staying online
- Work is needed to move any existing saved cards across

Suggested order

1. Add the CVV field now, since payments are currently exposed to fraud
2. Then replace the card section with the payment provider's form, which fixes the rest together